

SSPL ERP

Backup, Restore, Update & Rollback — Installation and Usage Guide

ERPNext v16 | Ubuntu Server 24.04 + Docker Compose | July 2026

Contents

1. Overview & server layout
2. Prerequisites
3. Part 1 — Backup system: installation
4. Part 1 — Backup system: daily use
5. Part 1 — Restoring from a backup
6. Part 1 — Optional cloud upload (rclone)
7. Part 2 — Update & rollback system: installation
8. Part 2 — Updating the system
9. Part 2 — Rolling back an update
10. Part 2 — Managing image backups
11. Scheduled jobs summary
12. Quick command reference
13. Troubleshooting

1. Overview & server layout

This toolkit has two independent parts:

- **Backup system** (`Backup/frappe_backup_system/`) — automated daily backups of the ERPNext **database, uploaded files (public and private), and configuration**, plus a restore script. Installed to `/opt/scripts/v2/`, backups stored in `/opt/backups/frappe/`.
- **Update & rollback system** (`Production Installation/update and rollback/`) — safely pulls new Docker images from GHCR, taking a full data backup and a Docker image snapshot first, so a bad update can be rolled back. Installed to `/opt/sspl-erp/v2/`.

Both parts install into their own `v2/` folder, **side-by-side** with any older scripts already on the server: nothing in `/opt/scripts/` or directly in `/opt/sspl-erp/` is replaced, so the old scripts remain available as a fallback. Backup *data* is shared — both generations store data backups in `/opt/backups/frappe/` and image snapshots in `/opt/sspl-erp/image-backups/`, so a backup made by one can be restored with the other.

```
Your server
├─ /opt/sspl-erp/
│  ├─ sspl-erp-*.sh          ← old update scripts (kept as fallback)
│  ├─ v2/
│  │  ├─ sspl-erp-common.sh  ← shared config: SITE_NAME lives here
│  │  ├─ sspl-erp-update-with-rollback.sh
│  │  ├─ sspl-erp-rollback.sh
│  │  └─ sspl-erp-backup-manager.sh
│  └─ image-backups/        ← Docker image snapshots (shared, rollback)
├─ /opt/scripts/
│  └─ v2/
│     ├─ frappe_backup.sh    ← full backup (DB + files + config)
│     ├─ frappe_db_backup.sh ← quick database-only backup
│     ├─ frappe_backup_verify.sh ← checks backups are complete & fresh
│     └─ frappe_restore.sh   ← restore a full backup
└─ /opt/backups/frappe/
   └─ 20260712_020000/
      ├─ *-database.sql.gz    ← database dump
      ├─ *-files.tgz          ← public files
      ├─ *-private-files.tgz  ← private files / attachments
      ├─ site_config.json, .env, docker-compose.yml
      └─ backup_manifest.txt
└─ db-only/                  ← 6-hourly database dumps
```

2. Prerequisites

- Ubuntu Server 24.04 with Docker Engine + Docker Compose v2 (`docker compose` , not `docker-compose`).
- The ERPNext stack deployed at `/opt/sspl-erp/` with a `docker-compose.yml` and a `.env` file containing `MARIADB_ROOT_PASSWORD` .
- Root / sudo access on the server.
- Your Frappe **site name**. To find it:

```
docker compose -f /opt/sspl-erp/docker-compose.yml exec backend ls sites/
```

The site name is the folder that is not `apps.txt` , `assets` , or `common_site_config.json` — for example `192.168.225.135` .

3. Part 1 — Backup system: installation

Step 1 — Copy the files to the server

```
# From the repo, copy either the folder or the tarball to the server, then:
tar -xzf frappe_backup_system.tar.gz
cd frappe_backup_system          # or: cd Backup/frappe_backup_system
```

Step 2 — Run the installer

```
chmod +x setup_frappe_backups.sh
sudo ./setup_frappe_backups.sh
```

The installer will:

1. Create `/opt/backups/frappe/` (permissions restricted to root — backups contain credentials).
2. Install the four scripts to `/opt/scripts/v2/` and make them executable — older scripts directly in `/opt/scripts/` are not touched.
3. **Ask for your site name** and write it into the backup, db-backup, and restore scripts.
4. Offer to install the cron jobs (see section 11 for the schedule).
5. Offer to run a test backup (answer `yes` the first time).

If the old backup scripts are already scheduled in cron: installing the new cron jobs too means every backup runs twice. Either answer `no` to the cron question while you test the v2 scripts manually, or say `yes` and then remove the old `/opt/scripts/frappe_*.sh` lines with `sudo crontab -e` once you are happy with v2.

Step 3 — Verify the test backup

```
sudo /opt/scripts/v2/frappe_backup_verify.sh
```

All four lines must say **OK**: database, public files, private files, and config. The script exits with an error if anything is missing.

4. Part 1 — Backup system: daily use

Once installed, backups run automatically from cron. Manual commands:

Task	Command
Full backup now (DB + files + config)	<code>sudo /opt/scripts/v2/frappe_backup.sh</code>
Quick database-only backup	<code>sudo /opt/scripts/v2/frappe_db_backup.sh</code>
Check backups are complete & recent	<code>sudo /opt/scripts/v2/frappe_backup_verify.sh</code>
See what is stored	<code>ls -lh /opt/backups/frappe/</code>
Watch the backup log	<code>tail -f /var/log/frappe_backup_v2.log</code>

Retention is automatic: full backups are kept **30 days**, database-only dumps **14 days**. To change this, edit `RETENTION_DAYS` at the top of the respective script in `/opt/scripts/v2/`.

Good habit: run a manual full backup before any risky change (new app install, configuration change, server maintenance) — and always before an update, although the update script also does this automatically.

5. Part 1 — Restoring from a backup

Warning: a restore **overwrites** the site's current database and files with the backup contents. Anything entered after the backup was taken is lost.

Step 1 — Pick a backup

```
ls /opt/backups/frappe/  
# e.g. 20260712_020000
```

Step 2 — Get the MariaDB root password

```
grep MARIADB_ROOT_PASSWORD /opt/sspl-erp/.env
```

Step 3 — Run the restore

```
sudo /opt/scripts/v2/frappe_restore.sh /opt/backups/frappe/20260712_020000
```

The script will:

1. Ask for the site name (press Enter to accept the configured default).
2. Ask for the MariaDB root password (input is hidden).
3. Ask for a final `yes` confirmation.
4. Restore the database, public files, and private files in one operation.
5. Run `bench migrate` (in case app versions differ from the backup) and restart all services.

Afterwards, log in to ERPNext and verify recent documents, attachments, and print formats.

6. Part 1 — Optional cloud upload (rclone)

By default backups stay on the server's own disk. To survive a disk failure, upload them to cloud storage. Full provider walkthrough: see *Rclone_Configuration_Guide.docx*.

Step 1 — Install and configure rclone (one time)

```
sudo apt install rclone  
rclone config      # create a remote, e.g. named "gdrive"
```

Step 2 — Point the backup script at the remote

Edit `/opt/scripts/v2/frappe_backup.sh` and set the variable near the top:

```
RCLONE_REMOTE="gdrive:frappe-backups"
```

Every full backup is then uploaded automatically after it completes. If the upload fails, the backup still exists locally and a warning is written to the log.

Step 3 — Test it

```
sudo /opt/scripts/v2/frappe_backup.sh  
rclone ls gdrive:frappe-backups | head
```

7. Part 2 — Update & rollback system: installation

Step 1 — Copy the four scripts

Copy these from `Production Installation/update and rollback/` into `/opt/sspl-erp/v2/` (create it with `sudo mkdir -p /opt/sspl-erp/v2`). Older `sspl-erp-*.sh` scripts directly in `/opt/sspl-erp/` stay untouched as a fallback; both versions share the same `image-backups/` snapshots.

- `sspl-erp-common.sh` — shared configuration (sourced by the others, not run directly)
- `sspl-erp-update-with-rollback.sh`
- `sspl-erp-rollback.sh`
- `sspl-erp-backup-manager.sh`

Step 2 — Set the site name

Edit `/opt/sspl-erp/v2/sspl-erp-common.sh` and set the variable at the top (this is the **only** place it needs changing):

```
SITE_NAME="192.168.225.135"
```

Step 3 — Make the scripts executable

```
cd /opt/sspl-erp/v2
chmod +x sspl-erp-update-with-rollback.sh sspl-erp-rollback.sh sspl-erp-backup-manager.sh
```

Note: the update script calls `/opt/scripts/v2/frappe_backup.sh`, so install Part 1 first.

8. Part 2 — Updating the system

When a new image has been published to GHCR (`ghcr.io/santhosh3279/sspl-erpNext`):

```
cd /opt/sspl-erp/v2
./sspl-erp-update-with-rollback.sh
```

What it does, in order:

1. Runs a **full Frappe data backup** (database + files). If this fails you are asked whether to continue.
2. Snapshots the current Docker images to `/opt/sspl-erp/image-backups/backup_TIMESTAMP.tar`.
3. Stops all services, prunes unused Docker resources, and pulls the latest images.
4. Starts services and **waits until the database and backend are actually ready** (up to 3 minutes).
5. Repairs MariaDB user grants, runs `bench migrate`, clears the cache.
6. Prints the new version and deletes image snapshots older than the 3 most recent.

If any step fails, the script stops immediately and prints rollback instructions.

Verify after every update

```
docker compose ps
docker compose logs --tail=50
docker compose exec backend bench version
docker compose exec backend bench --site 192.168.225.135 doctor
```

Then log in and check the key business flows (e.g. create a draft invoice, open a report, test printing).

9. Part 2 — Rolling back an update

If the system misbehaves after an update:

```
cd /opt/sspl-erp/v2
./sspl-erp-rollback.sh
```

The script shows the most recent image snapshot (size and date), asks for confirmation, stops the services, loads the saved images, restarts, repairs DB grants, and clears the cache.

Rolling back to a specific older snapshot

```
./sspl-erp-backup-manager.sh list
BACKUP_FILE=/opt/sspl-erp/image-backups/backup_20260701_090000.tar ./sspl-erp-rollback.sh
```

Important: rollback restores the Docker **images** only — not the database. If the failed update's migration changed the database structure, also restore the data backup that the update script took automatically (section 5). Its timestamp is printed at the start of the update.

10. Part 2 — Managing image backups

Image snapshots are several GB each. The update script already keeps only the last 3; the manager gives manual control:

Task	Command
List snapshots (marks the latest)	<code>./sspl-erp-backup-manager.sh list</code>
Keep newest 5, delete the rest	<code>./sspl-erp-backup-manager.sh clean 5</code>
Same, without confirmation (for cron)	<code>./sspl-erp-backup-manager.sh clean 5 --yes</code>
Delete all snapshots	<code>./sspl-erp-backup-manager.sh delete-all</code>
Check disk space	<code>df -h /opt/sspl-erp</code>

Note: when run from cron there is no terminal to confirm on, so `--yes` is required — without it the command exits with an error instead of deleting.

11. Scheduled jobs summary

Installed by the Part 1 setup script (view with `sudo crontab -l`):

Schedule	Job	Log file
Daily, 02:00	Full backup — <code>frappe_backup.sh</code>	<code>/var/log/frappe_backup_v2.log</code>
Every 6 hours	DB-only backup — <code>frappe_db_backup.sh</code>	<code>/var/log/frappe_db_backup_v2.log</code>
Sunday, 03:00	Backup verification — <code>frappe_backup_verify.sh</code>	<code>/var/log/frappe_backup_verify_v2.log</code>

Optional — weekly cleanup of image snapshots (add with `sudo crontab -e`):

```
0 2 * * 0 /opt/sspl-erp/v2/sspl-erp-backup-manager.sh clean 5 --yes >> /var/log/sspl-erp-backup-clean.log 2>&1
```

12. Quick command reference

I want to...	Run
Back up everything now	<code>sudo /opt/scripts/v2/frappe_backup.sh</code>
Back up only the database	<code>sudo /opt/scripts/v2/frappe_db_backup.sh</code>
Check my backups are OK	<code>sudo /opt/scripts/v2/frappe_backup_verify.sh</code>
Restore a backup	<code>sudo /opt/scripts/v2/frappe_restore.sh /opt/backups/frappe/<TIMESTAMP></code>
Update to the latest release	<code>cd /opt/sspl-erp/v2 && ./sspl-erp-update-with-rollback.sh</code>
Undo the last update	<code>cd /opt/sspl-erp/v2 && ./sspl-erp-rollback.sh</code>
See image snapshots	<code>./sspl-erp-backup-manager.sh list</code>
Free disk space	<code>./sspl-erp-backup-manager.sh clean 3</code>
Check services	<code>docker compose ps</code>
Check ERPNext version	<code>docker compose exec backend bench version</code>
View recent logs	<code>docker compose logs --tail=50</code>

13. Troubleshooting

Backup verification says something is MISSING

```
sudo tail -50 /var/log/frappe_backup_v2.log      # find the failing step
ls -lh /opt/backups/frappe/<latest>/             # see what was actually saved
sudo /opt/scripts/v2/frappe_backup.sh           # re-run manually and watch the output
```

Update failed partway through

The script prints instructions when it fails. In short: roll back the images with `./sspl-erp-rollback.sh`; if the migration had already modified the database, also restore the data backup taken at the start of the update (section 5).

Rollback cannot find or load a snapshot

```
ls -lh /opt/sspl-erp/image-backups/             # is the .tar there?
tar -tvf /opt/sspl-erp/image-backups/backup_*.tar | head # is it readable?
# Load manually if needed:
docker load -i /opt/sspl-erp/image-backups/backup_<TIMESTAMP>.tar
docker compose up -d
```

Site unreachable after update / rollback

```
docker compose ps                                # any container restarting or exited?
docker compose logs backend --tail=100
docker compose exec backend bench --site 192.168.225.135 doctor
```

Disk is filling up

```
du -sh /opt/sspl-erp/image-backups /opt/backups/frappe
./sspl-erp-backup-manager.sh clean 2             # image snapshots
# Data backup retention: lower RETENTION_DAYS in /opt/scripts/v2/frappe_backup.sh
```

Cron jobs not running

```
sudo crontab -l                                  # jobs present?
grep CRON /var/log/syslog | tail                 # did they fire?
sudo tail /var/log/frappe_backup_v2.log          # what did they say?
```